Profiling-Guided Adaptive Optimization for Memory-Bound LLM Inference via Data Movement Analysis

Eugene Won
Information Coding and Processing Lab
Department of Electronic & Electrical Engineering, AI Software Engineering
Ewha Womans University
Seoul, Republic of Korea
eugenewon12@ewhain.net

***Abstract*—Large language model (LLM) inference is increasingly dominated by memory bandwidth limitations rather than computational capacity. In particular, autoregressive decoding exhibits extremely low arithmetic intensity, making it inherently memory-bound and leading to severe underutilization of modern GPUs. While existing optimization techniques such as operator fusion and mixed precision improve performance, they are typically applied statically without considering workload-dependent bottlenecks.**

In this work, we present a profiling-guided framework that analyzes LLM inference from a data movement perspective and derives adaptive optimization strategies based on workload characteristics. By systematically measuring latency, GPU utilization, and memory-related behaviors across varying batch sizes and sequence lengths, we characterize bottleneck regimes and identify how different optimizations impact effective memory traffic. Based on these observations, we propose a simple yet effective adaptive policy that selects optimization strategies depending on runtime conditions. Our results demonstrate that workload-aware optimization improves inference efficiency compared to static configurations while providing insights into memory-centric AI system design.

I. Introduction

Recent advances in large language models (LLMs) have dramatically increased computational requirements, but real-world inference performance is often limited by memory bandwidth rather than compute capability. In particular, the autoregressive decoding phase exhibits very low arithmetic intensity, requiring repeated access to model weights stored in high-bandwidth memory (HBM). As a result, inference performance is fundamentally constrained by data movement rather than arithmetic operations.

Under practical deployment settings, such as batch size one or latency-sensitive applications, GPU utilization remains low despite high theoretical FLOPS. This inefficiency arises from a combination of kernel launch overhead, insufficient parallelism, and frequent memory access, which together lead to memory-bound execution.

Although optimization techniques such as mixed precision and operator fusion are widely used, they are typically applied uniformly without considering workload-dependent behavior. However, the dominant bottleneck shifts depending on factors such as batch size and sequence length.

In this work, we adopt a system-level perspective and analyze LLM inference through the lens of memory-centric AI. Rather than focusing solely on reducing computation, we investigate how different optimization strategies affect data movement and effective memory utilization.

II. Related Work

Prior work on accelerating LLM inference includes compiler-based optimizations such as `torch.compile`, mixed precision execution, and memory-efficient attention mechanisms like FlashAttention. These methods reduce kernel overhead and improve data locality.

Recent systems research has also emphasized the importance of memory-centric design, highlighting that modern AI workloads are often limited by memory bandwidth rather than compute. Techniques such as batching, tiling, and data reuse aim to increase arithmetic intensity and reduce memory traffic.

However, existing approaches generally apply optimizations statically and do not explicitly consider how bottlenecks vary across workloads. There is limited work that systematically connects runtime profiling, memory behavior, and adaptive optimization strategies.

III. Proposed Method

We propose a profiling-guided adaptive optimization framework that explicitly models memory-bound behavior and data movement.

A. Profiling and Bottleneck Characterization

We collect runtime metrics using PyTorch Profiler and GPU monitoring tools, including latency, kernel execution time, GPU utilization, and memory usage.

To better understand memory-bound behavior, we analyze workloads in terms of arithmetic intensity and data movement. Based on these metrics, we classify execution regimes into:

- **Kernel-overhead-bound:** Dominated by frequent kernel launches

- **Memory-bound:** Limited by memory bandwidth and data movement
- **Low-utilization regime:** Insufficient parallelism leading to idle compute units

B. Optimization Strategies

We evaluate practical system-level optimizations that influence data movement and execution efficiency:

- Operator fusion via `torch.compile` to reduce redundant memory access
- Mixed precision (FP16) to reduce memory footprint and bandwidth usage
- Batch size scaling to increase arithmetic intensity and parallelism

C. Adaptive Optimization Policy

Based on profiling results and workload characteristics, we derive a rule-based adaptive policy:

- Kernel-overhead-bound $\rightarrow$ apply operator fusion
- Memory-bound $\rightarrow$ reduce memory traffic via FP16
- Low-utilization $\rightarrow$ increase batch size to improve parallelism

This policy reflects the relationship between workload characteristics, data movement, and system performance.

IV. Experimental Plan

We conduct experiments using GPT-2 and lightweight LLMs under varying batch sizes and sequence lengths.

We compare:

- Baseline inference
- Mixed precision (FP16)
- `torch.compile`
- Profiling-guided adaptive optimization

We measure:

- Latency (ms/token)
- Throughput (tokens/sec)
- GPU utilization
- Memory usage (as a proxy for data movement)

Additionally, we analyze how performance changes as workloads transition from memory-bound to compute-bound regimes by varying arithmetic intensity through batch scaling.

V. Expected Contributions

This work provides:

- A memory-centric analysis of LLM inference bottlenecks
- An empirical study of how optimization techniques affect data movement
- A practical adaptive strategy for workload-aware optimization

Our approach emphasizes that improving AI system performance requires understanding and reducing data movement, not just computation.

VI. Conclusion

We propose a profiling-guided adaptive optimization framework for memory-bound LLM inference. By analyzing execution through data movement and arithmetic intensity, our approach improves inference efficiency while providing insights into memory-centric AI system design.

References

[1] PyTorch Team, "TorchDynamo and TorchInductor," 2023.

[2] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS, 2022.

[3] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP, 2023.